

PROFESSIONAL SUMMARY

“ 성공 케이스보다, 실패 케이스를 고려하여, 우리 시스템이 어떻게 버틸지부터 설계합니다.

SRT 철도 예매 및 EV 충전 플랫폼에서 결제·정산 코어 시스템의 설계 및 구현을 주도했습니다. '외부 시스템 장애는 필연적'이라는 전제 하에 4단계 방어선(망취소-자동롤백-배치보정-수동개입)을 구축하여 외부의 실패가 내부 데이터 정합성 훼손으로 이어지지 않도록 방어했습니다. 철저히 얹힌 레거시 코드를 도메인 중심 구조로 재설계하여 복잡도를 낮추고, 장애가 발생해도 데이터가 틀어지지 않는 구조를 만드는 데 집중합니다.

- ✓ **Data Integrity:** 카드/포인트 복합 결제 1원 오차를 원천 차단하는 도메인 모델 설계
- ✓ **Operational Cost:** 결제 오류 원인을 로그만으로 1분 내 규명 가능한 Audit Trail 체계화
- ✓ **Legacy Modernization:** 블랙박스화된 외부 결제 모듈(C Lib)을 JNA로 직접 제어하여 장애 통제권 확보

TECHNICAL SKILLS

ARCHITECTURE

Payment Orchestration

Fail-Safe Design

ISO 8583

Spring Batch

BACKEND & DATA

Java 17/21

Spring Boot 3.x

Oracle / Tiberio

Redis

JPA / QueryDSL

PROTOCOL &
SECURITY

Signal Protocol (E2EE)

Netty

JNA (Java Native Access)

WebRTC

WORK EXPERIENCE

VitzroSys (비츠로시스)

2025.04 — 현재

결제/정산 도메인 리드 (Core Module Design)

- SRT 통합 결제 플랫폼: 10개 이상의 VAN/PG사 연동 규격을 표준화한 **Payment Adapter Pattern**을 설계하여 신규사 연동 기간을 2주에서 3일로 단축했습니다.
- 무결성 보장: 결제/취소 트랜잭션을 Command 객체로 캡슐화하여, 실패 시 원자적(Atomic) 롤백을 보장했습니다.
- 운영 효율화: 모호한 에러 메시지를 단계별 코드로 세분화하여, 운영팀의 장애 인지 및 전파 속도를 개선했습니다.

Investy (인베스티)

2022.10 — 2025.10

백엔드 코어 개발 (Security & Legacy Migration)

- 보안 관제 시스템: 네트워크 차단 환경을 우회하는 **Signal Server**를 독자 구축하여 클라우드 벤더 락인(Vendor Lock-in)을 해소했습니다.
- EV 충전 플랫폼: Maven/Legacy Spring을 **Spring Boot 3.x**로 마이그레이션하여 빌드 및 배포 속도를 50% 단축했습니다.
- 비용 최적화: Google Maps API 의존성을 제거한 자체 위치 DB(Spatial Index) 구축으로 월 운영비 1,000만 원을 절감했습니다.

EDUCATION

2022.02 — 2022.09 한국 소프트웨어 인재 개발원: Java 기반 웹 개발자 심화 과정 수료

2017.09 — 2017.12 한국 소프트웨어 기술 진흥 협회: IoT 및 임베디드 시스템 과정 수료

Career Description

주요 프로젝트별 기술적 의사결정 및 트레이드오프 상세

[SRT] 차세대 통합정보시스템 결제/정산 플랫폼

2025.04 — 현재

ROLE 결제/정산 코어 전담 개발 (초기 설계부터 운영 이관까지)

TECH STACK Java 17, Spring Boot 3, Oracle, Spring Batch

1. 복합 결제 1원 오차 방어를 위한 금액 도메인 모델 설계

PROBLEM 카드/포인트 복합 결제 후 부분 취소 시, PG사와 내부 시스템 간 **소수점 처리 방식 차이**로 인한 **정산 불일치** 발생 리스크.

SOLUTION 서로 다른 금액 데이터를 일관된 구조로 다루기 위해 **금액 표준화 모델**을 설계했습니다. 결제 수단별 잔액과 분배 책임을 가지는 **Allocation 모델**을 통해 사용 가능 금액을 자동 검증하고, 필요한 금액을 순차적으로 이동시켜 자금 정합성을 확보했습니다.

FAIL-FAST "마이너스 잔액은 절대 존재할 수 없다"는 불변식을 생성자에 강제하여, 잘못된 계산 로직이 DB에 커밋되기 전 즉시 Exception을 발생시켜 **데이터 오염을 원천 차단**했습니다.

2. 외부 연동 실패를 전제로 한 Fail-Safe 아키텍처

FAILURE PG사 승인은 성공했으나 내부 DB 저장 실패(Dual Write 문제) 시, **고객 돈은 나가고 표는 없는** 치명적 상황 발생.

SOLUTION [요청 기록 → 승인 요청 → 응답 기록 → 확정] 4단계 상태 전이 로직을 구현했습니다. 내부 저장 실패 시 즉시 **망취소 (Net-Cancel)**를 수행하고, 망취소 실패 시 배치 재시도를 통해 미확정 거래가 자동 복구되도록 다중 방어 체계를 설계했습니다.

3. 레거시(ISO 8583) 통신 및 역분개 정산 시스템

ACTION **FEP(Front End Processor)** 레벨의 TCP 소켓 모듈을 직접 구현하여 전문 파싱 로직을 내재화했습니다. 취소 시 원래 데이터를 삭제하지 않고, **상쇄 데이터(-금액)**를 적재하는 **역분개 방식**을 적용하여 모든 자금 흐름을 사후 추적할 수 있도록 설계했습니다.

4. 외부 연동 암호화 규격 불일치 해결

ISSUE 페이코 연동 시 공식 문서(AES-128)와 실제 응답 패킷이 달라 복호화 실패.

ACTION 로그 단위 비교 분석을 통해 문서에 명시되지 않은 **고정 IV 값**과 **AES-256 사용**을 역추적해냈습니다. 이를 기반으로 공식 가이드 수정을 요청하고 연동을 정상화했습니다.

1. Signal Server 독자 구축 (Cloud 종속성 제거)

CONSTRAINT 외부 클라우드 불모지 및 망 분리 환경에서 **Signal Protocol의 E2E 암호화** 구현 필요.

SOLUTION 오픈소스 Signal Server의 AWS 종속성(DynamoDB, SQS)을 제거하고, **Redis와 MySQL 조합으로 경량화 재구축**했습니다. 수천 개의 세션 키(PreKey) 관리 로직을 최적화하여 단일 서버에서도 안정적인 메시지 라우팅 처리가 가능하도록 구현했습니다.

2. 네트워크 차단을 우회하는 터널링 아키텍처

PROBLEM 특정 국가의 **DPI(Deep Packet Inspection)** 시스템이 메신저 패킷 패턴을 탐지하여 차단.

SOLUTION 앱과 서버 간 통신을 네트워크 차단 환경에서도 유지하기 위한 **커스텀 터널링 구조**를 설계하여 서버 IP를 은닉하고 트래픽을 난독화했습니다.

TRADE-OFF 터널링으로 인해 레이턴시가 **평균 200ms 증가**했으나, '전송 속도'보다 '**연결 생존성**'이 핵심 요구사항이었으므로 이를 수용했습니다.

⚡ 한화모티브 전기차 충전 플랫폼 리팩토링

2023.12 — 2024.10

1. Legacy Refactoring 및 의존성 격리

LEGACY XML 설정 기반 Spring Legacy와 외부 업체 C 라이브러리가 혼재되어 수정 시 사이드 이펙트 예측 불가.

SOLUTION **Spring Boot 3.x**로 전환하며 Java Config로 설정을 명시화했습니다. 외부 라이브러리 호출부는 **Interface로 추상화**하여 격리하고, 실제 구현체는 JNA로 연결하여 비즈니스 로직과 기술 코드를 분리했습니다.

2. JNA 기반 결제 모듈 내재화

ISSUE 구형 리더기(EUC-KR)와 신형 서버(UTF-8) 간 **인코딩 충돌**로 결제 실패 빈번.

ACTION **JNA(Java Native Access)**를 통해 C 메모리 영역에 직접 접근하여 인코딩 변환 브릿지를 구현했습니다. 블랙박스였던 결제 프로세스를 내재화함으로써 장애 대응 시간을 수 일에서 수 분 단위로 단축했습니다.

3. WebClient 기반 비동기 추천 엔진

LOGIC [광고 > 추천 > 최저가 > 거리] 순의 **Waterfall 추천 알고리즘**을 적용했습니다. **WebClient**의 Non-blocking I/O를 활용하고, 타임아웃 발생 시 기본값(거리순)으로 Fallback 처리하여 장애 확산을 방지했습니다.

비용 절감을 위한 Geo-Spatial DB 자체 구축

COST

Google Maps API 호출 비용 과다로 서비스 확장에 걸림돌.

ACTION

행정구역 폴리곤 데이터를 **자체 DB(Spatial Index)**에 구축하여 API 의존성을 제거, **월 1,000만 원 비용 절감**을 달성했습니다.